

Function Point Analysis: Difficulties and Improvements

CHARLES R. SYMONS

Abstract—The method of Function Point Analysis was developed by Allan Albrecht to help measure the size of a computerized business information system. Such sizes are needed as a component of the measurement of productivity in system development and maintenance activities, and as a component of estimating the effort needed for such activities. Close examination of the method shows certain weaknesses, and the author proposes a partial alternative. The paper describes the principles of this “Mark II” approach, the results of some measurements of actual systems to calibrate the Mark II approach, and conclusions on the validity and applicability of function point analysis generally.

Index Terms—Complexity, estimating, function points, maintenance, productivity, system development.

I. INTRODUCTION

THE size of the task of designing and developing a business computerized information system is determined by the product of three factors (see Fig. 1).

- The *information processing size*, that is some measure of the information processed and provided by the system.
- A *technical complexity factor*, that is a factor which takes into account the size of the various technical and other factors involved in developing and implementing the information processing requirements.
- *Environmental factors*, that is the group of factors arising from the project environment (typically assessed in project risk measures), from the skills, experience and motivation of the staff involved, and from the methods, languages, and tools used by the project team.

The first two of these factors are intrinsic to the size of the system in the sense that they result directly from the requirements for the system to be delivered to the user.

Allan Albrecht has described a method known as “Function Point Analysis” for determining the relative size of a system based on these first two factors [1]–[4]. The method has gained widespread acceptance in the business information systems community, for system size assessment as a component of productivity measurement, when system development or maintenance and enhancement activities are completed. Where historic productivity data are available, the method can also be used as an aid in estimating man-hours, from the point where a functional requirements specification is reasonably complete.

Manuscript received June 28, 1985.

The author is with Nolan, Norton & Company, One Lumley Street, London W1Y 1TW, England. (Nolan, Norton & Co. is an information technology firm of KPMG Peat Marwick.)

IEEE Log Number 8718288.

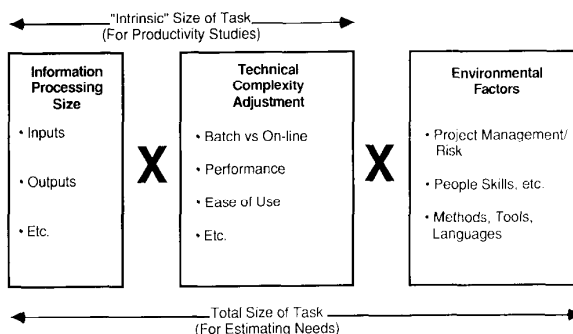


Fig. 1. The three components of system size.

For estimating purposes, the third group of environmental factors clearly also has to be taken into account.

The aims of this paper are:

- to critically review function point analysis,
- to propose some ways of overcoming the weaknesses identified,
- to present some initial results of measurements designed to test the validity of the proposed improvements.

While the paper contains some criticisms of Albrecht's Function Point method, the author wishes to acknowledge the substantial contribution made by Albrecht in this difficult area. The ideas in this paper are an evolutionary step, only made possible by Albrecht's original lateral thinking.

However as Albrecht's method becomes ever more widely adopted, it will become a de facto industry standard. Before that happens it is important to examine the method for any weaknesses, and if possible to overcome them.

II. ALBRECHT'S FUNCTION POINT METHOD

In Albrecht's Function Point Method, the two components of the intrinsic size of a system are computed as summarized in the following:

- 1) The Information Processing Size is determined by first identifying the system components as seen by the end-user, and classifying them as five types, namely the external (or “logical”) inputs, outputs, inquiries, the external interfaces to other systems, and the logical internal files. The components are each further classified as “simple,” “average,” or “complex,” depending on the number of data elements in each type, and other factors. Each component is then given a number of points depending on

Description	Level of Information Processing Function			Total
	Simple	Average	Complex	
External Input	x 3 = _____	x 4 = _____	x 6 = _____	_____
External Output	x 4 = _____	x 5 = _____	x 7 = _____	_____
Logical Internal File	x 7 = _____	x 10 = _____	x 15 = _____	_____
Ext. Interface File	x 5 = _____	x 7 = _____	x 10 = _____	_____
External Inquiry	x 3 = _____	x 4 = _____	x 6 = _____	_____
Total Unadjusted Function Points				_____

(a)

ID	Characteristic	DI	ID	Characteristic	DI
C1	Data Communications	_____	C8	On-line Update	_____
C2	Distributed Functions	_____	C9	Complex Processing	_____
C3	Performance	_____	C10	Re-Useability	_____
C4	Heavily Used Configuration	_____	C11	Installation Ease	_____
C5	Transaction Rate	_____	C12	Operational Ease	_____
C6	On-Line Data Entry	_____	C13	Multiple Sites	_____
C7	End User Efficiency	_____	C14	Facilitate Change	_____
Total Degree of Influence				_____	

DI Values

Not Present, or No Influence = 0

Insignificant Influence = 1

Moderate Influence = 2

Average Influence = 3

Significant Influence = 4

Strong influence, Throughout = 5

$$TCF = 0.65 + 0.01 \times (\text{Total 'Degree of Influence'})$$

(b)

Fig. 2. (a) Unadjusted FP counting. (b) Technical Complexity Factor.

its type and complexity [see Fig. 2(a)] and the sum for all components is expressed in "Unadjusted Function Points" (or UFP's).

2) The Technical Complexity Factor is determined by estimating the "degree of influence" of some 14 component "General Application Characteristics" [see Fig. 2(b)]. The degree of influence scale ranges from zero (not present, or no influence) up to five (strong influence throughout). The sum of the scores of the 14 characteristics, that is the total degrees of influence (DI), is converted to the "Technical Complexity Factor" (TCF) using the formula

$$TCF = 0.65 + 0.01 \times DI.$$

Thus each degree of influence is worth 1 percent of a TCF which can range from 0.65 to 1.35.

3) The intrinsic relative system size in Function Points (FP's) is computed from

$$FP's = UFP's \times TCF.$$

Function Points are therefore dimensionless numbers on an arbitrary scale.

Albrecht's reasons for proposing Function Points as a measure of system size are stated (4) as:

- the measure isolates the intrinsic size of the system from the environmental factors, facilitating the study of factors that influence productivity,
- the measure is based on the user's external view of the system, and is technology-independent,
- the measure can be determined early in the development cycle, which enables Function Points to be used in the estimation process, and
- Function Points can be understood and evaluated by nontechnical users.

Another implied aim [2] is for a method which has an acceptably low measurement overhead.

III. FP ANALYSIS—A CRITICAL REVIEW

The following questions and difficulties have arisen in teaching and applying Albrecht's method.

A. Information Processing Size (Unadjusted FP's)

• Classification of all system component types (input, outputs, etc.) as simple, average, or complex, has the merit of being straightforward, but seems to be rather oversimplified. A system component containing, say, over 100 data elements is given at most twice the points of a component with one data element.

• The choice of "weights" [i.e., points per component type, see Fig. 2(a)] has been justified by Albrecht as reflecting "the relative value of the function to the user/customer" [3] and "was determined by debate and trial." It seems a reasonable question to ask if the weights obtained by Albrecht from his users in IBM will be valid in all circumstances, and useful for size measurement in both productivity assessment and estimating. Some more objective assessment of the weights seems advisable. The weights also give rise to some surprising effects. Why for example should an inquiry provided via a batch input/output combination gain more than twice as many points as the same inquiry provided on-line? Why should interfaces be of any more value to the user than any other input or output?

• Most of the UFP's for a system arise from its externally seen inputs, outputs, interfaces, and inquiries. Differences in *internal* processing complexity between systems are reflected in two ways. First, the classification of any input, output, inquiry, or interface as simple, average

or complex depends in part on the number of logical file-types referenced from that component. As we shall see below, the latter is roughly related to internal processing complexity. (More recently guidelines have appeared [5] suggesting that "entities" should be counted rather than logical files.) Second, internal complexity appears as one of the 14 "General Application Characteristics," and can thus contribute up to 5 percent to the Technical Complexity Factor. All in all, the way internal complexity is taken into account seems to be rather inadequate and confused. Systems of high internal complexity studied by the author do not appear to have had their size adequately reflected by the FP method in the opinion of the designers of those systems.

- Finally, Function Points do not appear to be "summable" in the way one would expect. For example, if three systems, discrete but linked by interfaces, are replaced by a single integrated system providing the same transactions against an integrated database, then the latter system will score less FP's than the three discrete systems. (Arguably the integrated system should score more FP's because it maintains data currency better.) This result arises partly because the FP counting rules credit an interface to both the issuing and receiving system, and partly because the integrated file will generally score less than the three discrete files.

One of Albrecht's findings from studies of productivity has been a falloff in productivity by a factor of roughly 3, as system size increases from 400 to 2000 FP's [3]. This is a very important result if true, but the finding depends on the accuracy with which FP's reflect relative system size. Most of the above criticisms of the FP method point toward the conclusion that the FP scale underweighs systems which are complex internally and have large numbers of data elements per component, relative to simpler and therefore "smaller" systems. If the criticisms are valid and significant, then the falloff in productivity with system size may not be as serious as apparently observed. Clearly it is an important issue to resolve.

B. The Technical Complexity Factor

- The restriction to 14 factors seems unlikely to be satisfactory for all time. Other factors may be suggested now, and others will surely arise in the future. A more open-ended approach seems desirable. Also some of the factors, as currently defined, appear to overlap (e.g., those concerned with application performance, a heavily used hardware configuration for processing, and a high transaction rate); some reshuffling of the factors appears desirable.

- The weights ("degree of influence") of each of the 14 factors are restricted to the 0-5 range, which is simple, but unlikely to be always valid. One obvious example is the factor which reflects whether the system is intended for multisite implementation. In practice, initial development of such a system can easily cost a great deal more than it would if intended only for a single site. A re-examination of the TCF weights is therefore also desirable.

In summary the Albrecht FP method and especially the weights, were developed in a particular environment. With the benefit of experience and hindsight various questions arise about the validity of the method for general application, especially for the information processing size component. The GUIDE Project Team [5] has made steady progress in clarifying the detailed rules for FP counting to make them easier to apply, but has not questioned the underlying principles or weights. In the next section we will return to basic principles to develop an alternative approach for the information processing size component, and in Section V we will examine the calibration of the new UFP measure against practical data, and re-examine the calibration of Albrecht's Technical Complexity Factor. The resulting alternative approach will be referred to as the "Mark II" Function Point method, to distinguish it from Albrecht's method.

IV. A NEW MEASURE OF INFORMATION PROCESSING SIZE ("MARK II" FUNCTION POINTS)

Given that we want a measure of Information Processing Size which is independent of technology (if that is possible) and is in terms which the system user will easily recognize, we will start with certain basic assumptions, namely:

- we will regard a system as consisting of logical "transaction-types," that is, of logical input/process/output combinations.
- Interfaces at this logical level will be treated as any other input or output. (If an input or output happens to go to or come from another application, and that fact increases the size of the task, then it should be reflected in the Technical Complexity Factor.)
- Inquiries will be considered just as any other input/process/output combination.
- The concept of a "logical file" is almost impossible to define unambiguously, particularly in a database environment, and at this level, the concept of "file" is not appropriate. The concept which correctly belongs at the logical transaction level is the "entity," that is anything (object, real, or abstract) in the real world *about which* the system provides information.

The other basic starting point is to establish the criterion which we will use to establish the size scale. As our aim is to obtain sizes which can be used for productivity measurement and estimating purposes, we will take the system size scale to be related explicitly to the *effort* to analyze, design, and develop the functions of the system. This is in contrast to Albrecht's aim of having a size which represents the "value" of function delivered to the user. The latter seems to be a more subjective criterion, and therefore less easy to verify or calibrate in practice.

The task then is to find properties of the input, process, and output components of each logical transaction-type which are easily identifiable at the stage of external design of the systems, are intelligible to the user, and can be calibrated so that the weights for each of the components are based on practical experience.

The most difficult component for which we need a size parameter is the process component. For this we rely on the work of McCabe [6] and others who have developed measures of software process complexity and shown for example that their measures correlate well with the frequency of errors found in the software. Such complexity measures are typically concerned with structural complexity, that is, they count and weight branches and loops. Sequential code between the branches and within loops does not add to complexity in this view. At the external design stage we do not know the processing structure of each logical transaction-type, and in any case this would be too complicated to assess and keep within the aims of method.

We do know however, from Jackson [7], that a well-structured function should match the logical data structure, which at this level is represented by the access path for the transaction through the system entity model. Since each step along the access-path through the entity model generally involves a selection or branch, or (if a one-to-many step) a loop, it seems a reasonable hypothesis that a measure of processing complexity is to count *the number of entity-types referenced by the transaction-type*. ("Referenced" means created, updated, read, or deleted.)

The above is a rather tenuous argument, and the result is a crude, first-order measure. Fig. 3 shows an entity model for a simple order-processing system with a few logical transactions, and the number of entities referenced per transaction. Examples like this, and the experience of counting entity references per transaction in real systems, support this measure as a plausible hypothesis.

For the other (input and output) components of each logical transaction-type, we will take the number of data element types as being the measure of the size of the component. This is on the grounds that the effort to format and validate an input, and to format an output is in the first-order proportional to the number of data elements in each of those components, respectively. Fig. 3 also shows illustrative numbers of input and output data element types for the order-processing transactions.

The net result of the above is that the Mark II formula for Information Processing Size expressed in Unadjusted Function Points becomes:

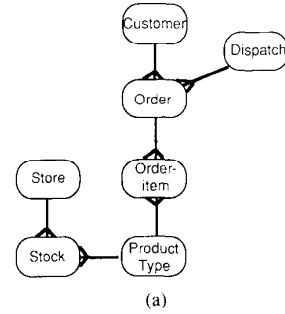
$$\text{UFP's} = N_I W_I + N_E W_E + N_O W_O$$

where

- N_I = number of input data element types,
- W_I = weight of an input data element type,
- N_E = number of entity-type references,
- W_E = weight of an entity-type reference,
- N_O = number of output data element types,
- W_O = weight of an output data element type,

and N_I , N_E , and N_O are each summed over all transaction-types.

(From now on whenever we refer to transactions, inputs, outputs, data elements, and entities, etc., it will be



(a)

Transaction-Type	Input Data Elements	Entities Referenced		Output Data Elements
		Types	(Total)	
Add New Customer	53	Customer	1	3
Check Stock Availability	2	Product-Type Store, Stock	3	10
Process Order-Header	20	Customer Order, Dispatch	3	40
Process Order-Item	6	Order, Dispatch, Order-Item Product-Type, Store, Stock	6	14
Cancel Order-Item	2	Customer, Order, Order-Item Product-Type	4	15
Stock Report by Store & Product	1	Store, Product-Type Stock	3	21
Total	84		20	103

(b)

Fig. 3. Example analysis of an order processing system for Mark II unadjusted function points. (a) Entity model. (b) Logical transaction analysis.

understood that we are referring to "types" unless it is necessary to distinguish between "types" and "occurrences.")

The next task is to attempt to determine the weights by calibrating this formula against practical data.

V. CALIBRATION OF MARK II FUNCTION POINTS

A first test and rough calibration of the Mark II Function Point method has been carried out using data collected from two Clients ("A" and "B") in consultancy studies. In each case the objective of the study was to explore the use of function point analysis for productivity measurement, and in particular the merits of the Mark II versus the Albrecht method.

Both Clients selected six systems for assessment, which were of varying size and technology. No constraint was placed on system selection, other than that the system should be of a size requiring at least 3-4 months to develop, and that an expert should be available to explain the system.

Collection of data for each system and its analysis fell into three categories, namely

- Unadjusted Function Point data
- Technical Complexity Factor data
- development effort data to calibrate the Mark II method.

Since the last of these three categories has parts in common with each of the first two, it will be described first.

A. Analysis of Development Effort Data

Of the 12 systems, 9 had been developed recently, and adequate data were available about the development effort for further analysis.

For calibration purposes, the project representatives were asked to analyze the man-hours which had been used for development, and break them down as shown in Fig. 4. ("Development man-hours" were defined strictly according to Albrecht's rules.)

The first split required for each system is between the man-hours which had been devoted to the pure Information Processing Size, that is the effort devoted to analysis, design, and development purely to meet user requirements, and the man-hours needed for the work on the various parts of the Technical Complexity Factor. The effort devoted to the Information Processing Size was further broken down into the effort required to handle input, processing, and output, defined as follows.

Input	Data entry, validation, error correction.
Processing	Performing the updates and calculations from availability of valid input until output data are ready for formatting.
Output	Formatting and transmission of output to the output device.

This required breakdown of development man-hours is unusual. No records were available, and therefore the breakdowns given are subjective. The project representatives did not demur from the task however; percentage breakdown splits between input, process, and output varied considerably, examples given including 35/10/55, 25/65/10, 40/40/20, etc. The validity of this approach can only lie in its statistical basis. Data from the nine systems analyzed so far appear to behave reasonably, as will be seen below. As data from more systems are collected so the quality and credibility of the derived weights will improve.

The man-hours apportioned to the Technical Complexity Factor were similarly further broken down by spreading them across the 14 Albrecht factors, and other factors proposed by the author (see below). This "top-down" split had the possibility of some "bottom-up" cross-checking by the project representatives, although still subjectively, as again no records were available. For example it might result from a first breakdown that two man-days were apportioned to a particular TCF factor. At this bottom level, project representatives could often recall how much effort really went into this particular factor. So with some iteration the size of the TCF component of development man-hours and its breakdown over the various factors was refined.

It should be emphasized that apart from prompting by the author on the required way of analysis, the development effort breakdown estimates are entirely those of the project representatives, working independently of each other, and without knowledge of the analysis which was to follow.

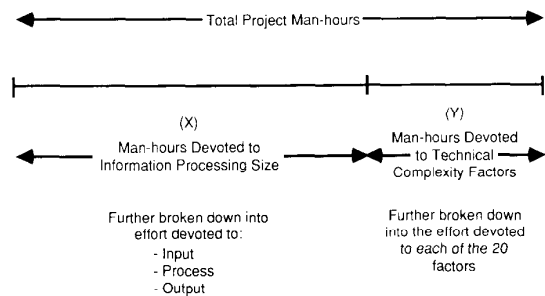


Fig. 4. The analysis of project development time.

B. Collection and Analysis of UFP Data for Albrecht and Mark II Methods

All 12 systems were assessed according to the Albrecht and Mark II methods. First, for the Mark II method, an entity model of the system was derived, and at this point the "logical internal files" for Albrecht's method were identified and scored. Then each system was broken down into its logical transactions. The components of each transaction were classified for complexity and scored according to Albrecht's rules, and in parallel the counts of input and output data elements and entity references were collected for the Mark II method.

Early in the course of this work it became apparent that some counting conventions and definitions were needed for the Mark II method to ensure consistency and simplicity, such as have been developed by GUIDE [5] and Albrecht [4] in their "Current Practice" chapters. Space does not permit a full account of these conventions, and they may well evolve further. An outline of the main types of conventions is given in the Appendix.

With the total counts of input and output data elements, and entity references for all transactions in each system, such as illustrated in Fig. 3, and the man-hours of development effort derived as in Section V-A above, it was possible to calculate the "man-hours per count" data shown for all nine systems in Fig. 5.

When examining Fig. 5, one must bear in mind the variety of system types, sizes and technologies, and of environments involved, and the crudity of the estimates of the breakdown of development effort over input, process and output. One should particularly note the sensitivity of some of the component man-hour estimates to a shift of a few percent between one category, e.g., input, and another, e.g., entity references.

In spite of these variations and uncertainties however, there is a clear pattern, and certain of the exceptionally high or low figures are explicable in terms of known project characteristics (the ringed figures in Fig. 5).

To get a "normative" set of weights for combined Client A and B data, averages were taken of all the non-ringed figures in Fig. 5. These are

1.56	man-hours per input data element
5.9	" " " entity reference
1.36	" " " output data element.

Client	System	Development Man-hours per Count			Technology
		Input Data Elements	Entity Refs	Output Data Elements	
A	1	1.30	12.9	1.31	Mainframe, Batch
	2	1.68	3.9	0.81	Mainframe, On-line
	3	1.67	3.0	0.97	Mini, On-line
	4	0.37 ¹	8.0	2.17	Mainframe, Batch
B	7	0.52	0.25	0.21 ²	PC with DB Handler
	8	0.72	4.1	0.35 ³	Mainframe, On-line
	9	1.3/	5.1	0.35 ³	Mainframe, Mainly Batch
	10	2.4	3.2	0.87 ²	Mainframe On-line
	11	1.8	7.2	1.45	Full-Safe Mini, On-line

- Notes
1. This system obtained its input from another system. Little effort was required for input formatting and validation
 2. This PC based system was unusual in producing very large numbers of documents with comparatively small variations across all document-types
 3. This client had implemented special mainframe software to reduce the effort of preparing output

Fig. 5. Estimated man-hours per count for nine development projects.

These figures may be used directly as the weights in the Mark II UFP formula and could also have some value for future estimating purposes.

However, in order that the Mark II method produces UFP's comparable to Albrecht's, the Mark II scale was "pegged" to Albrecht's by scaling down the above weights so that the average system size in UFP's for all 8 systems under 500 UFP's came out to be identical on both scales.

The Mark II formula for Information Processing Size on the basis of this data therefore becomes

$$\text{UFP's} = 0.44N_I + 1.67N_E + 0.38N_O.$$

The UFP sizes for all 12 systems, calculated according to this formula, were then plotted against the corresponding Albrecht UFP's (see Fig. 6).

Two conclusions may be drawn in interpreting this graph. First, there is a general tendency for the Mark II method to give a larger information processing size relative to Albrecht's, as system size increases. More data are needed to confirm this trend, but the first results are in the direction expected from taking into account internal processing complexity in the Mark II method.

The second conclusion is that the Mark II method shows its sensitivity, especially in smaller systems, to relatively high or low average numbers of data elements per input or output component. The asterisked system in Fig. 6 (system 11 in Fig. 5) is unusual in having many transactions with exceptionally low counts of data elements per input and output relative to the amount of processing it carries out.

As to the Clients' interpretation of these data, Client A considered that the relative sizes of its systems and the derived productivity data were more plausible on the Mark II scale than on Albrecht's. Client B did not have sufficient feeling for the relative sizes to choose between the two methods. This Client did however prefer the Mark II method of analysis as giving more insight into the size measurement process.

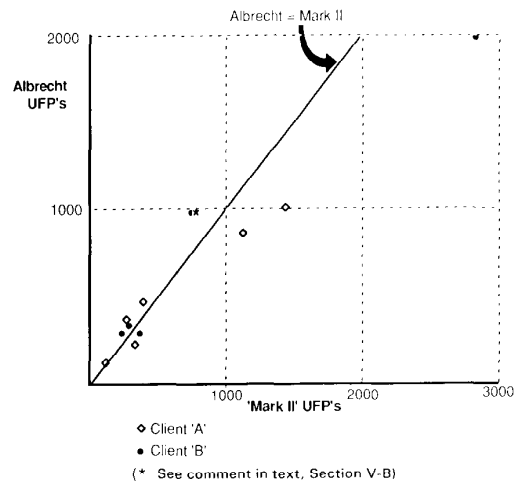


Fig. 6. Albrecht versus Mark II unadjusted function points comparison.

C. Collection and Analysis of TCF Data According to Albrecht and Mark II Methods

The "Degree of Influence" of the 14 Albrecht Technical Complexity Factor components was scored for each system, using the scoring guidelines described by Albrecht [4] and Zwanzig [5]. A further five factors were proposed by the author, and project representatives were invited to nominate any other factors which they felt ought to be included in this category. The additional five factors are the needs

- to interface with other applications (project representatives suggested this should be broadened to include interfaces to technical software such as message switching),
- for special security features,
- to provide direct access for Third Parties,
- for documentation requirements, and
- for special user training facilities, e.g., the need for a training subsystem.

An additional factor suggested by project representatives was the need to define, select and install special hardware or software uniquely for the application.

Considerable debate took place about the criteria for what can be counted as a TCF component. The rule which evolved is that a TCF component is a system requirement other than those concerned with information content, intrinsic to and affecting the size of the task, but not arising from the project environment.

In total therefore 20 factors were scored on Albrecht's "Degree of Influence" scale, and in addition, for the 9 development systems, the actual effort devoted to each of these 20 factors was estimated by the project representatives (see Section V-A).

Two analyses were performed to calibrate Albrecht's Degree of Influence scale against estimates of actual effort.

For the first analysis, the actual TCF was computed for each system from the formula

$$\text{TCF (actual)} = 0.65 (1 + Y/X)$$

where (see Fig. 4)

Y = man-hours devoted to Technical Complexity Factors

X = man-hours devoted to the Information Processing Size as measured by Unadjusted Function Points.

Fig. 7 shows the "TCF (actual)" plotted against the TCF derived from Albrecht's Degree of Influence scale for each of the 20 component factors. (Note the latter is not the pure Albrecht TCF, since Albrecht would only take 14 factors into account.)

In spite of the admitted roughness of the estimates going into the "TCF (actual)" figures, a clear pattern emerges from Fig. 7. First, Albrecht's method of assessing a TCF appears to work, but the weight of each Degree of Influence should vary with the technology. For the systems whose points lie close to the line labeled (2) in Fig. 7, a weight of 0.005 per Degree of Influence, or possibly less, is more appropriate than Albrecht's 0.01. In other words it seems to have taken less than half the effort to achieve these 20 technical complexity factors in practice than Albrecht's formula suggests. This correlates with the facts known about the systems lying along the line labeled (2). Client B has developed special software to simplify the development of on-line mainframe systems, while two of the other three systems along this line are personal computer based, and built with a fourth generation language ("Natural"), respectively. In contrast Albrecht's formula was derived from projects developed with technology available in the late 1970's.

The second more detailed analysis attempted to correlate the Degree of Influence scores of individual TCF components against the estimated actual percentage development effort. Owing to the small number (9) of projects, the 20 components, and the roughness of the estimates, no firm conclusions can be drawn about the relative weights of the individual components. The following are first indications, but much more data will be needed to firm up these indications.

- Some grouping of Albrecht's components seems desirable; the distinctions made between his components 3, 4, and 5 concerned with performance, and 6, 7, and 8 concerned with on-line dialogs, were not completely clear to the project representatives.
- Components 11 (installation ease), 12 (operational ease), 14 (ease of changes) and 16 (security) seem to require less effort per Degree of Influence than the other components.
- Component 19 (documentation) requires maybe double the effort per Degree of Influence of the other components.
- Component 9 (complex internal processing) does not strictly fit into the criteria for TCF components as now defined above. If this component is to stay, the guidelines for its scoring need to be more sharply defined.
- Component 13 (multiple-site implementation), as already noted, needs to be much more open-ended in its scoring.

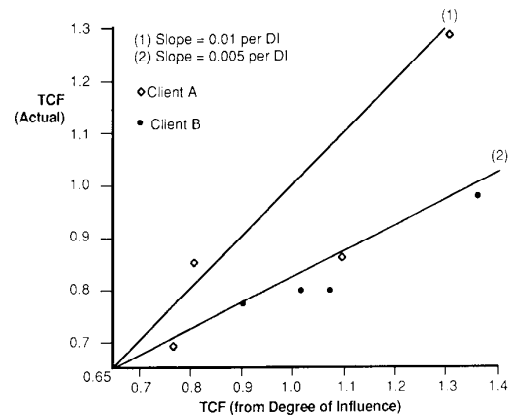


Fig. 7. Technical complexity factor comparison (actual versus degree of influence of 20 factors).

VI. CONCLUSIONS

The experience of applying Albrecht's Function Point Method and the alternative Mark II approach to a variety of systems has led to three groups of conclusions:

- 1) Albrecht versus Mark II Function Points.
- 2) Use of function points for productivity measurement and estimating.
- 3) Limitations of function points.

A. Albrecht Versus Mark II Function Points

The criticisms of Albrecht's Function Point method were given in Section III of this paper. The aim of the Mark II approach has been to overcome these weaknesses, particularly in the area of reflecting the internal complexity of a system. There will never be any "proof" that the Mark II approach gives superior results to that of Albrecht. Only the plausibility of the underlying assumptions, and judgement of many users on the results provided by both methods over a long period, will support one approach or the other.

As practitioners have gained experience in Albrecht's method in the last few years, it has evolved in the direction of the Mark II approach. First, the count of data elements has been introduced to make the complexity classification of inputs, outputs, etc., more objective. More recently the concept of "entities" has begun to replace "logical files." The remaining essential difference between the two methods is the way of looking at *data*. Albrecht's criterion for attributing UFP's to data is simply that the data is seen to exist in the system, regardless of how much of it is used; the Mark II approach attributes UFP's to data depending on its *usage* (create, update, read, delete) in transactions which form part of the system under study. Referring back to Albrecht's aims for Function Point Analysis, clearly it is the latter which is of "value" to the user. The *potential* value of stored data may be huge for a user, but it is only its actual value, resulting from use by transactions provided in applications, which can be measured by function points.

Other differences or similarities in practice between the two methods which are worthy of note are as follows.

• The Mark II approach requires an understanding of entity analysis, and rules are emerging (see Appendix) for entity counting conventions. For the Albrecht approach a knowledge of entity analysis is advisable, but no entity counting conventions have yet been published.

• The simplicity of the Mark II approach in having fewer variables than Albrecht's method in the UFP component has a number of advantages, such as greater ease of calibration against measurements or estimates, as shown in this paper. Also, if another hypothesis is made in the future for one of the UFP components, e.g., that the "size" of an input is proportional to, say, the number of data elements plus a constant, then it is easy to recalibrate the weights and explore the sensitivity of relative system sizes to the new hypothesis.

• There is a potential in the Mark II method for refining the measurement of the work-output in maintenance and enhancement activities which has not been tested so far. With Albrecht's method it is only possible to measure the total size of the changed system components. No distinction is possible between small and large changes to any single component. With the Mark II method, by recording the numbers of data elements changed, and counting references to entities which have been changed (or whose attributes have been changed), it should be possible to produce a measure of the size of the changes themselves (made to the changed components), that is, a measure more directly related to the work-output of maintenance and enhancement activities.

• The effort of counting data elements for each input and output means that FP measurement following the Mark II method may require 10–20 percent more effort than with Albrecht's method (which typically imposes about a one quarter percent overhead for system size measurement on the man-hours for a development project). Also the latter method may be applicable slightly earlier in the project life-cycle than the Mark II method, although it should be possible to produce reasonably accurate estimates of numbers of data elements per transaction for early sizing purposes.

B. Use of Function Points for Productivity Measurement and Estimating

An important conclusion illustrated by results from this study is that *function points are not a technology-independent measure of system size*, which was one of Albrecht's stated claims. The technology dependence is implicit in the weights used for the UFP and TCF components. This conclusion applies equally to the Albrecht and Mark II approaches. The conclusion for the TCF component is clearly illustrated in the results shown in Fig. 7. In Fig. 8 a hypothetical example is given for the size of error introduced into relative system size measurement by using an inappropriate set of weights, e.g., a set derived for a different technology from that actually being used.

A reasonable summary of the dependence of function points on technology is that the weights used by an or-

Suppose we have two systems, A and B, with the following characteristics

		N_I		N_E		N_O
	System A	100		20		100
	System B	100		20		20
	Weights (conventional technology)	0.5		2		0.4
Then	Size A	50	+	40	+	40 = 130
	Size B	50	+	40	+	8 = 98
					Ratio Sizes A/B	= 1.33

But, if technology changes, such that weights should be

Then	Size A	50	+	40	+	20	=	<u>110</u>
	Size B	50	+	40	+	4	=	<u>94</u>
								Ratio Sizes A/B = <u>1.17</u>

Conclusion: Use of the wrong set of weights may distort the ratio of sizes

Fig. 8. Example of sensitivity of function points to weights.

ganization in function point measurements imply a certain baseline or normative technology for that organization. If a system is built with a different technology, and its size is calculated with the organization's normative function point method, then the size calculated is the size the system would be if it were developed using the normative technology. This is clearly still very useful if the goal is to find out how productivity achieved with a new technology compares with that achieved with the normative technology. However

• if a new technology is introduced then sizes computed by function points cannot be reliably used for *estimating* unless a new set of weights is estimated or calibrated in line with the new technology, and

• if an organization changes its whole (normative) technology and it wants to continue to make fair size and productivity comparisons, then it must calibrate a new set of normative function point weights for the organization; clearly the latter will be necessary only at very infrequent intervals.

Generally for estimating purposes in an organization, a historical pool of information on actual productivity achieved using function point methods would be an invaluable asset. However, variations in environmental factors, especially project risk, the learning difficulties of new technologies and the performance of individuals can be extremely important considerations when estimating.

C. Limitations of Function Point Analysis

For completeness it is important to understand certain limitations of function point analysis.

• The method is not as easy to apply in practice as it first appears. In particular, working back from an installed physical system providing an interactive dialog to derive the logical transactions, requires some experience. Also cases sometimes arise where it becomes a matter of subjective judgement whether subtypes of a logical transaction (e.g., those which have slightly different processing paths depending on input values) are counted as separate logical transactions, or whether the differences can be ignored. For some time to come therefore it will be best in any one organization if all measurements are su-

pervised by one objective, experienced function point analyst. Such an analyst should accumulate and document cases and derive general rules such as given in the Appendix, which will help ensure consistency and objectivity in function point analysis.

- The suggestion has been made that it should eventually be possible to compute Mark II unadjusted function points automatically from a functional model of a system stored for example in a data dictionary. The difficulty with this is not so much automating the counting but, bearing in mind the previous limitation, ensuring that the model is correct and in a form suitable for FP counting. (However the benefits of this effort may be much wider than just the benefit of being able to count function points!)

- FP analysis works for installed applications, not for tools, or languages, such as a general purpose retrieval language. The distinction between these two classes of systems is not always absolutely clear. Applications provide preprogrammed functions where the user is invited to enter data and receives output data. Tools provide commands with which the user can create his or her own functions. Business information systems are usually applications, but may sometimes incorporate tools, or features with tool-like characteristics as well. Tools have a practically infinite variety of uses. The productivity of a group which supports a series of tools for use by others, for example an Information Center, can only be measured indirectly by sampling the productivity achieved by the end-users who apply the tools.

- A further limitation is that although in general FP analysis works for business applications it may not work for other types of applications, such as scientific or technical. This arises from its limited ability to cope with internal complexity. Again there is no absolute distinction between these different categories but internal complexity in business applications arises mainly from the processes of validation, and from interactions with the stored data. The Mark II method has aimed at reflecting these processes in a simple way. But scientific and technical applications typically have to deal with complex mathematical algorithms; FP analysis as currently defined has no reliable way of coping with such processes.

VII. CONCLUDING REMARKS

The function point method as proposed by Albrecht has certain weaknesses, but it appears they can be overcome by adjustments to the counting method as outlined here in the Mark II approach. These methods still seem to offer one of the best lines of approach for an organization that wishes to study its trends in productivity and improve its estimating methods for the development and support of computerized business information systems.

APPENDIX

SUMMARY OF ENTITY AND DATA ELEMENT COUNTING RULES

The following is a brief outline of the rules which have evolved, and which will probably evolve further, to simplify entity and data element counting, and to introduce more objectivity into the counting.

A. Entities

A distinction is made between three types of entities.

First, we count those entities with which the system is primarily concerned. These are typically the subjects of the main files or database of the system.

Second, we distinguish those things which data analysts frequently and rightly consider as entities, but about which the system holds typically only at most an identifying code, or name, and/or a description, and which are stored only to validate input. There are many possible rules to help distinguish such entities; information about them is usually held in files referred to as "system master tables" or "parameter tables." These are not included in our count of entity-references per transaction, on the grounds that their contribution to system size will be taken into account in the count of input data elements (which is considered to account for validation).

Third, entities about which information is produced only on output, for example in summary data reports, are also not counted. Their contribution to system size is considered to be reflected in the count of output data elements per transaction.

B. Data Elements

Several data element counting conventions are required; examples include:

- Data element *types* are counted, not data element occurrences.
- Conventions are needed for counting of dates (which may be subdivided), address fields (which may be multi-line), and the like.
- Batch report jobs, whether initiated by an operator, or automatically at certain points in time, are considered to have always at least a one-data element input.
- Field labels, boxes, underlines, etc., should be ignored.

ACKNOWLEDGMENT

The author acknowledges with gratitude the permission of the two Clients A and B to use their data for this paper. He also wishes to thank Client staff for their patience, support, and enthusiasm in the collection and analysis of the data.

REFERENCES

- [1] A. J. Albrecht, "Measuring application development productivity," in *Proc. IBM Applications Development Symp.*, GUIDE Int. and SHARE Inc., IBM Corp., Monterey, CA, Oct. 14-17, 1979, p. 83.
- [2] —, "Function points as a measure of productivity," in *Proc. GUIDE 53 Meeting*, Dallas, TX, Nov. 12, 1981.
- [3] A. J. Albrecht and J. E. Gaffney, "Software function, source lines of code and development effort prediction: A software science validation," *IEEE Trans. Software Eng.*, vol. SE-9, no. 6, pp. 639-647, Nov. 1983.
- [4] A. J. Albrecht, "AD/M Productivity Measurement and Estimate Validation—Draft," IBM Corp. Information Systems and Administration, AD/M Improvement Program, Purchase, NY, May 1, 1984.
- [5] K. Zwanzig, Ed., *Handbook for Estimating Using Function Points*, GUIDE Project DP-1234, GUIDE Int., Nov. 1984.
- [6] T. J. McCabe, "A complexity measure," *IEEE Trans. Software Eng.*, vol. SE-2, no. 4, pp. 308-320, 1976.
- [7] M. Jackson, *Principles of Program Design*. London: Academic, 1975.



Charles R. Symons received the B.Sc. degree in physics from Birmingham University, England, in 1959.

He is a Managing Consultant in the London office of Nolan, Norton and Company, where his main concern is helping large organizations develop strategies for investment in information systems in support of their business objectives. He also maintains a continuing interest in methods for the development and operation of business data processing systems, and the effective management of such activities. His career covers an association of some 25 years with

scientific and business computing, working for both Government and private organizations. His first computer programming was carried out around 1960 to analyze data collected in reactor physics experiments for the UK Atomic Energy Authority. In 1964 he joined CERN (Centre European pour la Recherche Nucleaire) in Geneva, Switzerland, and became Operations Manager of one of the largest scientific computer centres in Europe. Subsequently he joined Philips Electronics in London, and over 11 years undertook various assignments in business data processing. These included establishing the Corporate Information Systems Standards and Data Administration function at Philips Headquarters in Eindhoven, in The Netherlands. He has published other papers on various topics in physics, computer accounting, computer security, and data analysis.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.